AI LIFE OS
(AI-Powered Life Operating System)

By: Rumaysa Abdul Bari

Date: 29-04-2026

Table of Content

1. Introduction
2. Requirements
3. Construction
4. Working
5. Code
6. Conclusion & Future Scope

Introduction

Effective time management and productivity tracking have become major challenges in today's fast-paced academic and personal life. Students and individuals often struggle to manage multiple tasks, deadlines, and projects simultaneously. This leads to poor organization, missed deadlines, last-minute stress, and reduced overall efficiency.

To address this problem, AI Life OS (AI-Powered Life Operating System) is developed as an intelligent and user-friendly productivity dashboard. This application integrates task management, deadline tracking, project organization, and performance monitoring into a single platform. Unlike traditional task managers, it also includes an AI Assistant, which allows users to ask questions, get instant help, and improve productivity.

The system enables users to:

- Add and manage daily tasks
- Schedule deadlines using a calendar
- Track projects and their completion
- Monitor performance through visual graphs

- Earn points as motivation for completing tasks

By combining automation, visualization, and artificial intelligence, AI Life OS provides a structured and efficient way to manage daily activities. It reduces decision fatigue, improves consistency, and helps users stay organized.

Overall, this project demonstrates how modern technologies like Python, Streamlit, and AI can be used to build a practical and effective productivity system that enhances both time management and performance

Requirements

1. Physical Requirements

- Processor: Intel Core i3 or above / AMD equivalent
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: At least 500 MB free space for Python, libraries, and project files
- Display: Minimum 1280 × 720 resolution
- Internet Connection: Required for AI Assistant functionality
- Operating System: Windows 10/11, macOS, or Linux

2. Software Requirements

- Programming Language:
- Python 3.8 or higher
- Required Python Libraries:
- streamlit – For building the interactive web application
- matplotlib – For charts and data visualization
- groq – For AI Assistant integration
- streamlit-calendar – For calendar-based task scheduling
- Development Tools:
- Code Editor: Visual Studio Code / PyCharm / Notepad++ (VS Code recommended)
- Python Package Manager: pip (comes with Python)
- Web Browser: Google Chrome / Microsoft Edge / Mozilla Firefox

Construction

STEP 1 :Install Python

- Install Python 3.8+ from python.org and make sure Add to PATH is checked. Verify:

python --version
pip --version

STEP 2: Create project folder

- Make a folder named **ai-life-os** and open it in your code editor or File Explorer.

STEP 3: Add project folder

- Add files
- Inside **ai-life-os** create:
- **app.py**
- **requirements.txt**
- **users.jason (for storing user data)**

STEP 4: Create & activate virtual environment

- Windows:

cd "path\to\ai-life-os"
python -m venv venv
venv\Scripts\activate

STEP 5: Prepare requirements.txt

- Put this in requirements.txt:

streamlit
matplotlib
groq
streamlit-calender

STEP 6: Install dependencies

pip install -r requirements.txt

STEP 7: Add basic app code

- Open **app.py** and add the Streamlit app code • Login system
- Dashboard with charts
- Planner and tasks
- Deadlines and projects
- AI Assistant integration
- Save the file.

STEP 8:Add features

If you want extras later:

- Task management system
- Calendar-based scheduling
- Performance tracking graph
- Points system for motivation
- AI assistant for user queries

STEP 9:Run the app locally

- From the project folder:

  streamlit run app.py

STEP 10:Document & finalize

- Add screenshots and update **README.md** (intro, how to run, requirements, construction, future scope). When ready, initialize git and push to GitHub:

How to run

- Install Python (version 3.8 or above) from python.org and make sure it's added to your PATH.
- Download the project folder to your computer.
- Open a terminal/command prompt in the project folder.
- Install the required libraries:

pip install -r requirements.txt

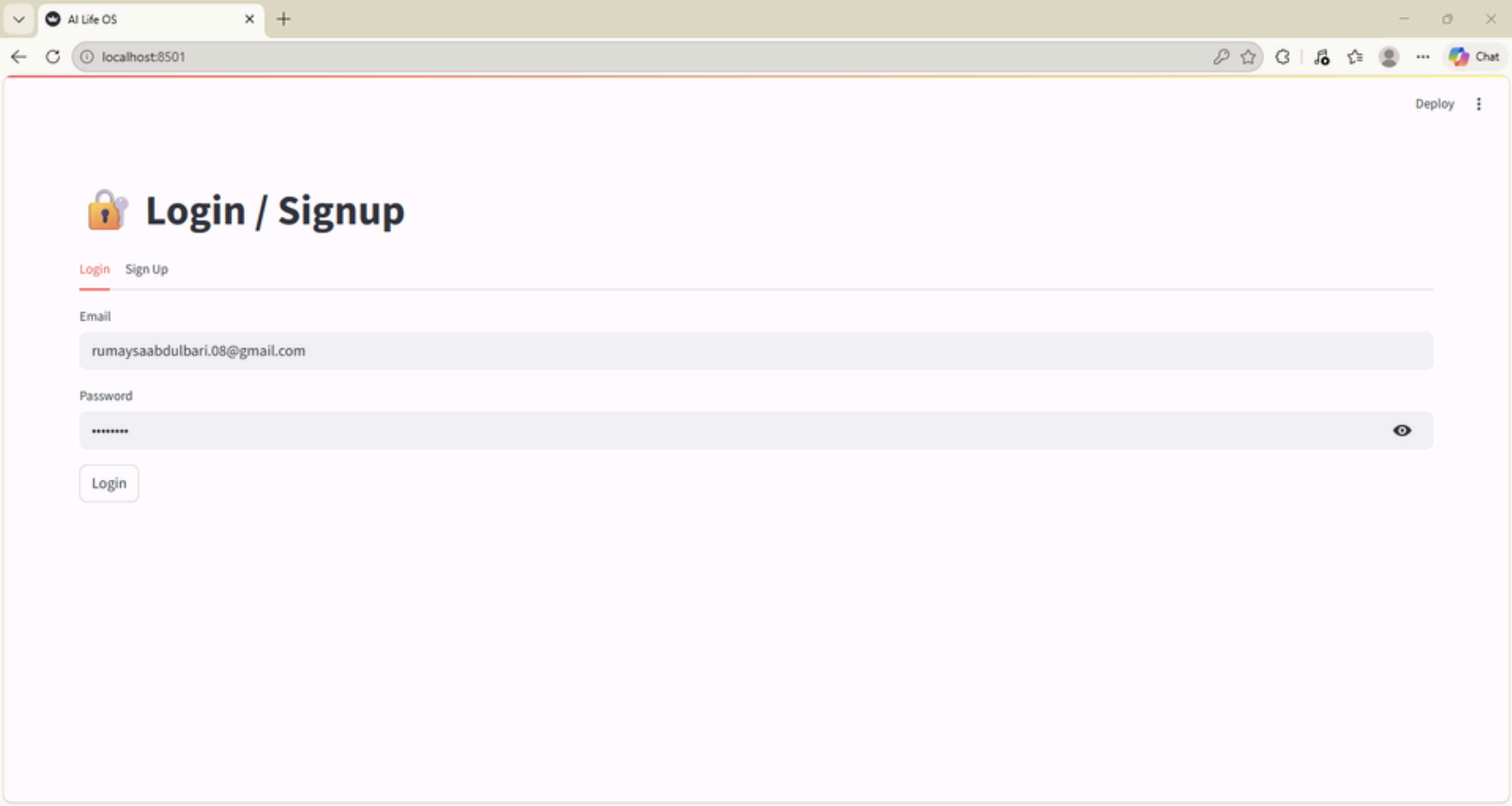
- Start the application:

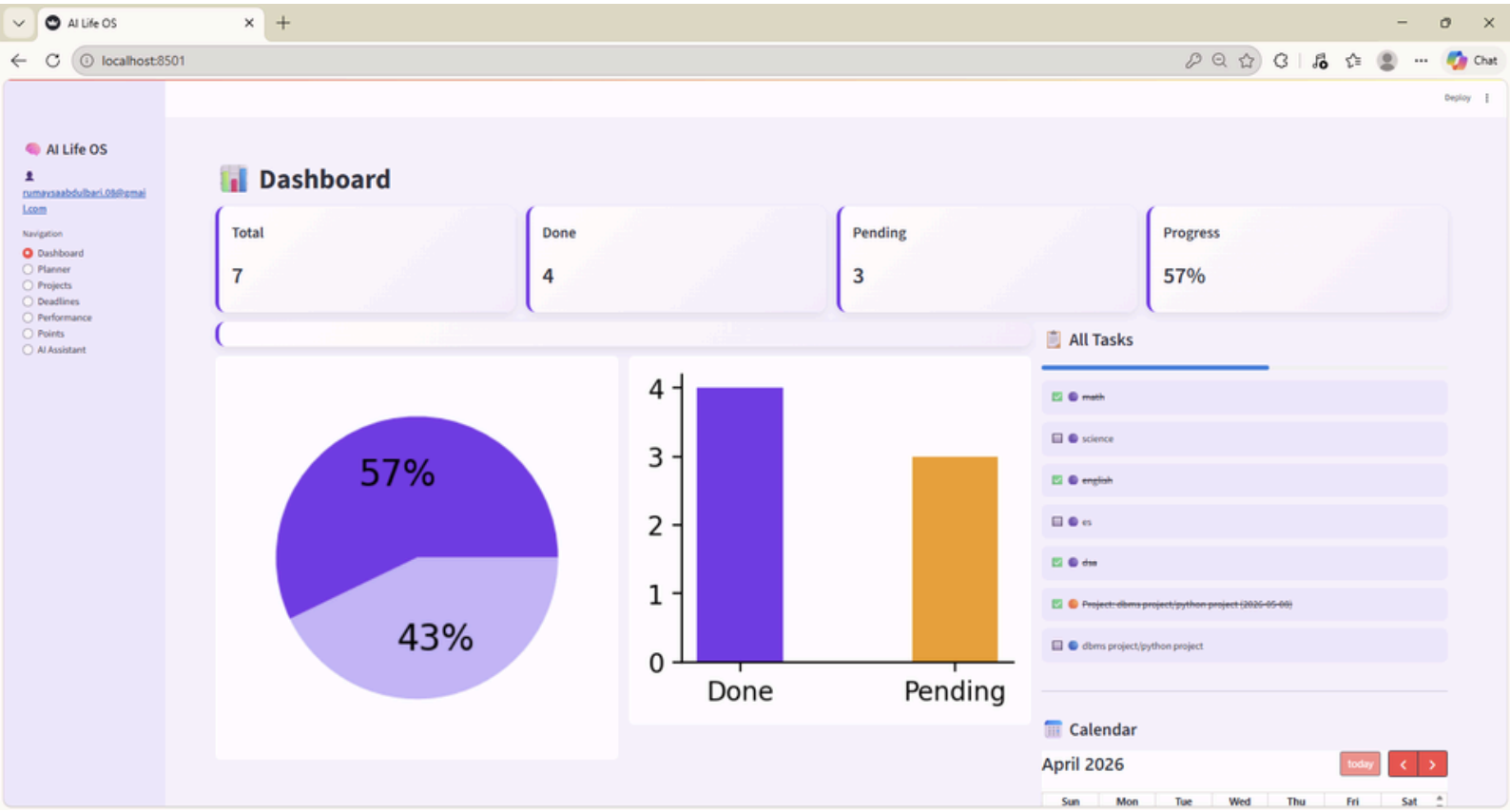
streamlit run app.py

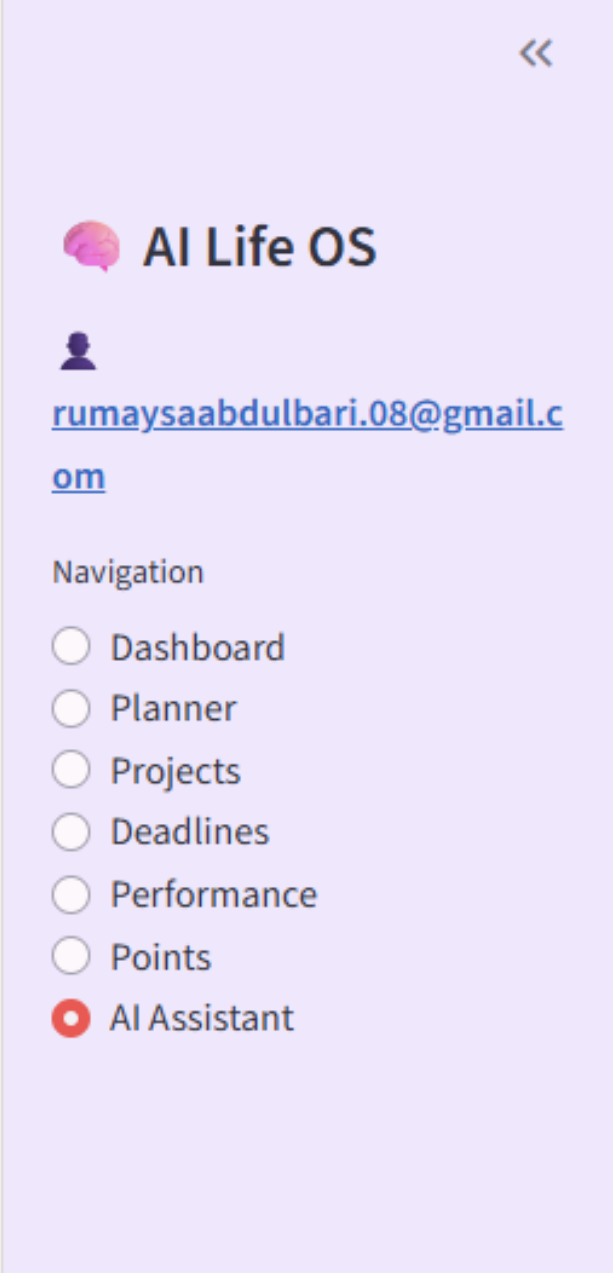
- The app will open automatically in your default browser

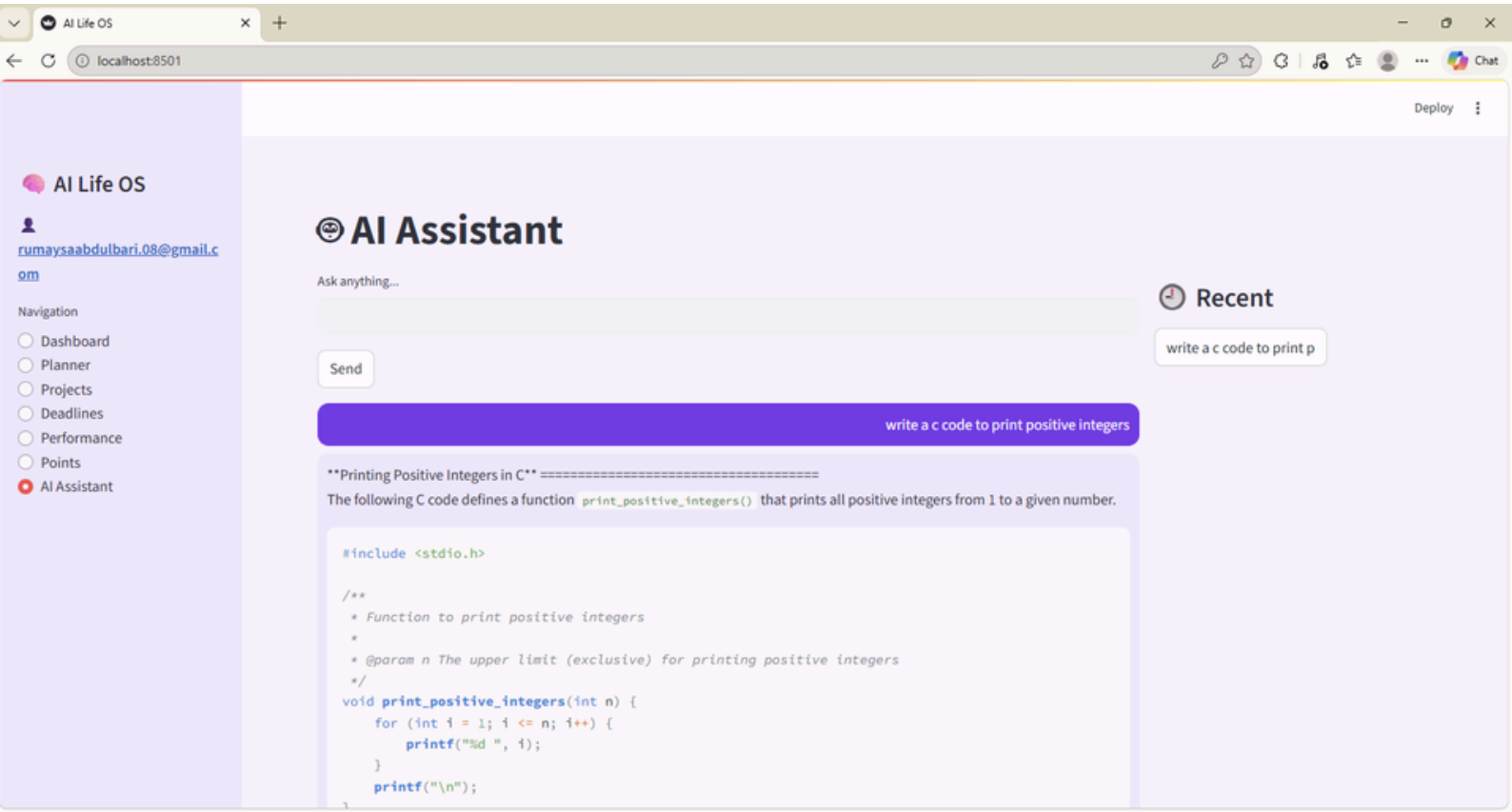
```
C:\WINDOWS\system32\cmd.

Microsoft Windows [Version 10.0.26100.4652]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rumay>cd "C:\Users\rumay\OneDrive\Desktop\study planner"

C:\Users\rumay\OneDrive\Desktop\study planner>python -m venv venv

C:\Users\rumay\OneDrive\Desktop\study planner>venv\Scripts\activate

(venv) C:\Users\rumay\OneDrive\Desktop\study planner>pip install -r requirements.txt
Collecting streamlit (from -r requirements.txt (line 1))
  Using cached streamlit-1.48.1-py3-none-any.whl.metadata (9.5 kB)
Collecting pandas (from -r requirements.txt (line 2))
  Using cached pandas-2.3.1-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting numpy (from -r requirements.txt (line 3))
  Using cached numpy-2.3.2-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting matplotlib (from -r requirements.txt (line 4))
  Using cached matplotlib-3.10.5-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting altair!=5.4.0,!=5.4.1,<6,>=4.0 (from streamlit->-r requirements.txt (line 1))
  Using cached altair-5.5.0-py3-none-any.whl.metadata (11 kB)
Collecting blinker<2,>=1.5.0 (from streamlit->-r requirements.txt (line 1))
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting cachetools<7,>=4.0 (from streamlit->-r requirements.txt (line 1))
  Using cached cachetools-6.1.0-py3-none-any.whl.metadata (5.4 kB)
Collecting click<9,>=7.0 (from streamlit->-r requirements.txt (line 1))
  Using cached click-8.2.1-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging<26,>=20 (from streamlit->-r requirements.txt (line 1))
  Using cached packaging-25.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pillow<12,>=7.1.0 (from streamlit->-r requirements.txt (line 1))
  Using cached pillow-11.3.0-cp313-cp313-win_amd64.whl.metadata (9.2 kB)
Collecting protobuf<7,>=3.20 (from streamlit->-r requirements.txt (line 1))
  Using cached protobuf-6.31.1-cp310-abi3-win_amd64.whl.metadata (593 bytes)
Collecting pyarrow>=7.0 (from streamlit->-r requirements.txt (line 1))
  Using cached pyarrow-21.0.0-cp313-cp313-win_amd64.whl.metadata (3.4 kB)
Collecting requests<3,>=2.27 (from streamlit->-r requirements.txt (line 1))
  Using cached requests-2.32.4-py3-none-any.whl.metadata (4.9 kB)
Collecting tenacity<10,>=8.1.0 (from streamlit->-r requirements.txt (line 1))
  Using cached tenacity-9.1.2-py3-none-any.whl.metadata (1.2 kB)
Collecting toml<2,>=0.10.1 (from streamlit->-r requirements.txt (line 1))
```

```
C:\Users\rumay>cd C:\Users\rumay\OneDrive\Desktop\Rumaysa Personal\GitHub\ai-life-os

C:\Users\rumay\OneDrive\Desktop\Rumaysa Personal\GitHub\ai-life-os>streamlit run app.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.0.4:8501
```


AI Life OS
localhost:8501
Deploy
Login / Signup
Login
Sign Up
Email
rumaysaabdulbari.08@gmail.com
Password
Login


AI Life OS
localhost:8501
AI Life OS
rumaysaabdulbari.08@gmail.com
Navigation
Dashboard
Planner
Projects
Deadlines
Performance
Points
AI Assistant
Dashboard
Total
7
Done
4
Pending
3
Progress
57%
57%
43%
4
3
2
1
0
Done
Pending
All Tasks
math
science
english
es
dss
Project: dbms project/python project (2026-05-08)
dbms project/python project
Calendar
April 2026
today
Sun
Mon
Tue
Wed
Thu
Fri
Sat


AI Life OS
rumaysaabdulbari.08@gmail.com
Navigation
Dashboard
Planner
Projects
Deadlines
Performance
Points
AI Assistant


AI Life OS
localhost:8501
Deploy
AI Life OS
rumaysaabdulbari.08@gmail.com
Navigation
Dashboard
Planner
Projects
Deadlines
Performance
Points
AI Assistant
AI Assistant
Ask anything...
Send
Recent
write a c code to print p
write a c code to print positive integers
Printing Positive Integers in C ==========
The following C code defines a function print_positive_integers() that prints all positive integers from 1 to a given number.
#include <stdio.h>
/**
* Function to print positive integers
*
* @param n The upper limit (exclusive) for printing positive integers
*/
void print_positive_integers(int n) {
for (int i = 1; i <= n; i++) {
printf("%d ", i);
}
printf("\n");

Syntax Code

1. User Login System

```
if st.button("Login"):
    if email in st.session_state.users and
st.session_state.users[email]["password"] == password:
        st.session_state.logged_in = True
        st.session_state.user = email
        st.rerun()
```

2. Task Addition (Planner)

```
if st.button("Add") and task:
    st.session_state.tasks.append({"name": task, "done":
False})
    save_current_user()
    st.rerun()
```

. Dashboard Calculation Logic

```
total = len(all_tasks)
done = sum(1 for t in all_tasks if t["done"])
percent = int((done / total) * 100) if total > 0 else 0
```

4. Project Completion & Points System

```
if new_val and not st.session_state.project_status[p]:
    st.session_state.points += 15
if not new_val and st.session_state.project_status[p]:
    st.session_state.points -= 15
```

5. Deadline Task Handling

```
if st.button("Add Task") and task:
    st.session_state.calendar_tasks[key].append(task)
    save_current_user()
    st.rerun()
```

6. AI Assistant Integration

```
response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system", "content": "You are a
helpful assistant."},
        {"role": "user", "content": user_input}
    ]
)
```

7. Saving User Data (Persistence)

```
def save_current_user():
    st.session_state.users[user]["tasks"] =
st.session_state.tasks
    st.session_state.users[user]["chat"] =
st.session_state.chat
    save_users(st.session_state.users)
```

Conclusion & Future Scope

The AI Life OS project demonstrates how modern technology can be used to solve real-world productivity challenges. By combining Python and Streamlit, the application provides an interactive and user-friendly platform that helps users manage tasks, track deadlines, organize projects, and monitor performance efficiently. The integration of an AI Assistant further enhances the system by providing instant support and improving user experience. Overall, the project serves as both a practical productivity tool and a strong demonstration of software development concepts.

Looking ahead, the project has significant potential for further improvement. Future versions could include integration with cloud databases for better data storage, real-time notifications and reminders for deadlines, and enhanced AI capabilities for personalized suggestions. Additional features such as mobile compatibility, multi-user collaboration, and advanced analytics can make the system more powerful and accessible.

In conclusion, AI Life OS successfully achieves its objective of creating a smart and efficient productivity system. With continuous enhancements and advanced features, it has the potential to evolve into a comprehensive digital life management platform that combines productivity, analytics, and artificial intelligence in a single solution.

Thank you